



Module 1: Automata, Computability & Complexity

Arnold B. Galve, DIT
abgalve@kld.edu.ph



Learning Outcomes

- ◆ Know (some) limitations of computer science.
- ◆ Understand the concept of computable and non-computable functions.
- ◆ Introduce the concept of Turing machines.
- ◆ Present ideas about halting.
- ◆ Enumerate example of tractable and non-tractable problems.

Limitations of Computer Science

Major reasons useful calculations cannot be done:

- ◆ Execution time of program is too long
 - problem can take years or centuries to finish
- ◆ Problem is not computable
 - no computer program can solve the problem
- ◆ We do not know how to write a program to solve the problem
 - problems that can conceivably be solved
 - includes many problems in artificial intelligence and computer vision, such as understanding language, object recognition, tracking, prediction

What problems can be solved by computers?

- ◆ Equivalent to: “What decision problems can be solved?”
- ◆ Decision problem
 - problem where the answer is always YES/NO
- ◆ Arbitrary problem can always be reduced to a decision problem
- ◆ There are decision problems for which algorithms do not exist
 - there are more different decision problems than there are different programs
 - if there are more decision problems than programs, then it must be the case that some decision problems cannot be solved by programs

Function Evaluation

- ◆ A decision problem is a question with a YES/NO answer:
 - Is 613,511 a prime number? **NO**
 - Is my birthday on a Sunday next year? **NO / YES**
 - Will I pass the Automata final exam?
- ◆ Sometimes a particular answer from a set of possible solutions is required
 - What is the smallest prime factor of 613,511?
 - On what day of the week is my birthday next year?
 - What grade will I get on the Automata final exam?
- ◆ The process of answering questions where a particular answer is uniquely determined from the given information can be viewed as the **evaluation of a function.**

Functions

- ◆ A **function** is a correspondence between a collection of possible *input values* and a collection of *output values* such that each possible input is assigned a unique output
- ◆ Example: the addition function
 - inputs are value pairs, outputs are values representing the sum of each input pair
 - output is unique, exactly one value for each input pair
- ◆ Process of determining output for given input is called **computing** or **evaluating** the function
- ◆ The ability to evaluate functions gives us the ability to solve problems
 - to solve the addition problem we must evaluate the addition function

- ◆ Example: $F(x) = x+1$

- ◆ $F(2) = 3$

- ◆ $F(x,y) = x+y$

- ◆ $F(2,3) = 5$

Computable Functions

- ◆ Algorithms are the means by which we formally express the evaluation of a function
- ◆ Can we design algorithms to evaluate **ALL** functions?
 - No!
- ◆ There are functions so complex that there is no well-defined step-by-step process for determining their output based on their input values
- ◆ The evaluation of these functions lies beyond the capabilities of any algorithmic system
- ◆ These functions are said to be **noncomputable**.
- ◆ Functions whose output values can be determined algorithmically from their input values are said to be **computable**.
 - functions having an algorithm that solves them

Limitations of Machines

- ◆ Machines can only perform computations described by algorithms:
 - the study of computable functions is the study of the ultimate capabilities of machines
- ◆ If we can build a machine with the capabilities to evaluate the entire set of computable functions, then the machines we build are as powerful as we can make them.
- ◆ On the other hand:
 - if we discover that the solutions to a problem requires the evaluation of a noncomputable function, we know the solution to that problem lies beyond the capabilities of machines

“Does ExtraTerrestrial life exist?”

- ◆ Q: Is it possible to write a Java program to answer the following question: *Does extraterrestrial life exist?*
- ◆ A: Yes!



- ◆ The question is answered either by a program that returns “yes” or by a program that returns “no”.
- ◆ We just don’t know which program is correct.

Is a problem Computable?

- ◆ Suppose we create a list of all programs for evaluating computable problems.
- ◆ To ask whether a problem is computable is to ask if there is a program in the list to solve the problem.
- ◆ This is not the same as actually identifying the program in the list that solves the problem.
- ◆ There are many computable problems whose solutions have not yet been identified.

Abstract Model of Computation

- ◆ The first abstract model of computation was defined by **Alan Turing** in 1936
 - The **Turing Machine**
- ◆ A Turing Machine is a purely theoretical device, developed well before technology was capable of providing the machines we know as “computers” today
- ◆ Model to study the capabilities and limitations of computational processes and computing machinery
- ◆ Provide a context in which solutions to all things computable can be expressed
- ◆ Provide a standard to which the power of other computational systems can be compared



Turing-Computable $\Leftarrow$ Computable

- ◆ **A function is computable if it can be computed by a Turing machine**
 - or, a function is computable if a program exists that can compute it
 - or, a function is computable if there is an effective algorithm for evaluating the function which, given any input values for which the function is defined, *produces the correct result and halts in a finite amount of time*
- ◆ **Q:** Is there anything a Turing Machine cannot do?
- ◆ **A:** Yes!

Noncomputability

- ◆ A function is **noncomputable** if there is no turing machine that could compute it
- ◆ or, a noncomputable function is any function that cannot be computed by any program
 - many more functions than there are programs
 - not possible to have a program for every function

A noncomputable problem

- ◆ Problem: List all subroutines that inputs an integer and return an integer

- here is one short subroutine:

```
int sub1(int x)
{
    return 1;
}
```

- and here is another:

```
int sub2(int x)
{
    return 2;
}
```

- ◆ Can make infinitely many such subroutines

Halting

```
int programA( int x )
{
    while( x == x )
        x = x;
    return x;
}
```

```
int programB( int x )
{
    while( x > 10 )
        x = x;
    return x;
}
```

- ◆ Programs that run forever do not halt
 - **programA**
 - programs with infinite loops
- ◆ Programs may halt on some inputs but not on others
 - **programB**
- ◆ Programs with no loops always halt
 - programs composed of assignment statements, **if** statements and the **return** statement

Halt or not?

- ◆ Will the following code eventually terminate?

```
while (x > 1)
{
    if (x > 2)
        x = x - 2;
    else
        x = x + 2;
}
```

- ◆ What if x is 8? How about 9?

Halt or not?

- ◆ Will the following code eventually terminate?

```
while (x > 1)
{
    if ( x % 2 == 0)
        x = x / 2;
    else
        x = 3*x + 1;
}
```

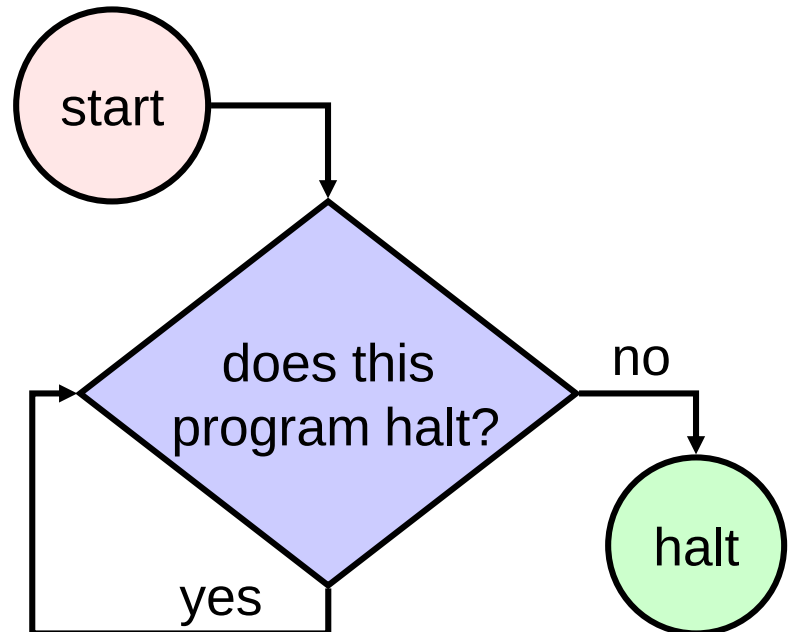
- ◆ What if **x** is 8? How about 7? How about any number > 0?

The Halting Problem

- ◆ **Q:** Is it possible to write a Java program to answer the following question

Does a given program halt on all inputs?

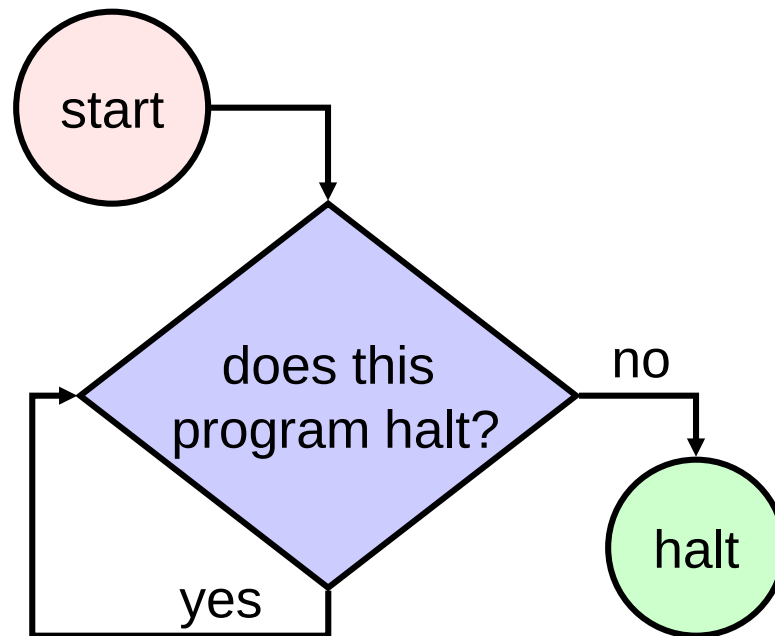
- ◆ It would be useful if we could write a program or compiler to test if a given source program contains an infinite loop
- ◆ If we had such a program, then we would never execute a program with an infinite loop
- ◆ This problem is known as the **halting problem**



The halting problem is noncomputable

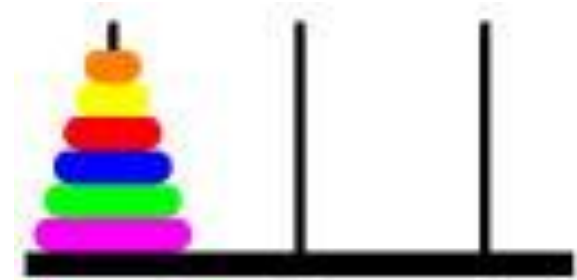
No finite program can be written that will check other programs and halt in a finite time giving a solution to the halting problem

- ◆ There is no single finite program that will answer the halting question for all programs

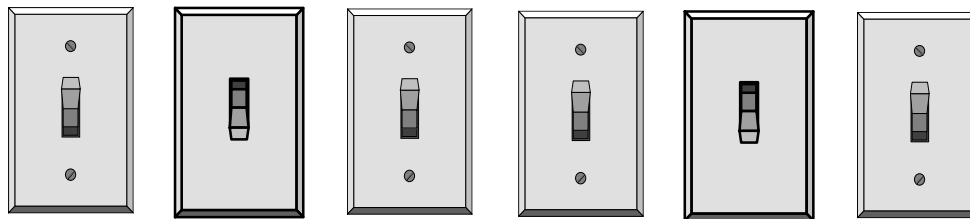


Not impossible, but impractical

- ◆ Towers of Hanoi
 - how long to move n disks?



- ◆ What combination of switches turns the light on?
 - try all combinations, how many are there?



Program Execution Time

- ◆ How long a program takes to perform computations on large inputs
- ◆ Computable problems are basically divided into two classes
- ◆ Computations that can be completed in a reasonable time period are called **tractable**
 - a computation is tractable if its running time on N inputs is logarithmic or polynomial in N
- ◆ **Intractable** computations are those that cannot be realistically completed except for small examples
 - a computation is intractable if its running time on N inputs is exponential in N

Examples

◆ Tractable problems

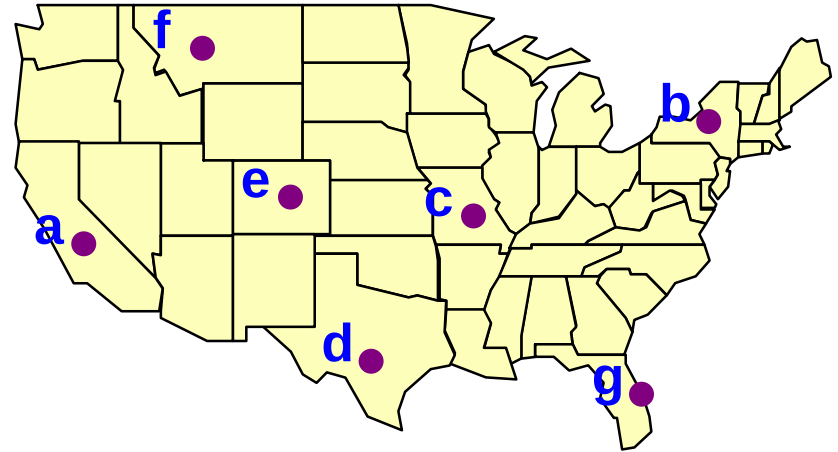
- searching or sorting **N** inputs
- computing the sum of positive integers of size **N**
- finding the largest element in an **N x M** array
- counting the number of characters in a Java program

◆ Intractable problems

- computing all permutations of **N** inputs
- Towers of Hanoi
- Traveling Salesperson
- Bin Packing
- Graph Coloring

Traveling Salesperson

- ◆ A traveling salesperson must visit each of some number of cities before returning home
 - knows the distance between each of the cities
 - wants to minimize the total distance traveled while visiting all of the cities
- ◆ In what order should the salesperson visit the cities?

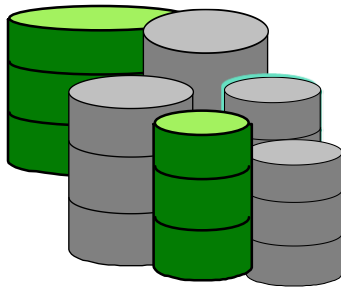


Try all paths, from every starting point -- how long does this take?

{a, b, c, d, e, f, g},
{b, a, c, d, e, f, g}, ...

Bin Packing

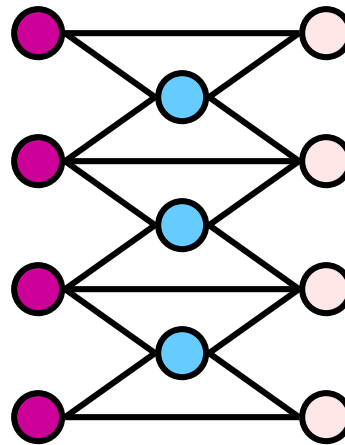
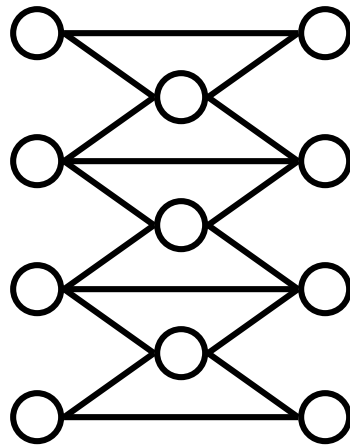
- ◆ Pack a given set of items having different sizes into a minimum number of equal-sized bins



- ◆ Example:
 - Pack trucks with barrels using the smallest possible number of trucks

Graph Coloring

- ◆ Color the **vertices** of a graph such that no **edge** connects two identically colored vertices using the minimum number of colors





Module 1: Automata, Computability & Complexity

Arnold B. Galve, MIT, DITc
abgalve@gmail.com

